

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана**

(национальный исследовательский университет)»

ФАКУЛЬТЕТ _____ **(МГТУ им. Н.Э. Баумана)**
«Информатика и системы управления»

КАФЕДРА _____ **«Теоретическая информатика и компьютерные технологии»**

Лабораторная работа № 8
по курсу «Алгоритмы компьютерной графики»
«Шейдеры OpenGL»

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

Москва 2025

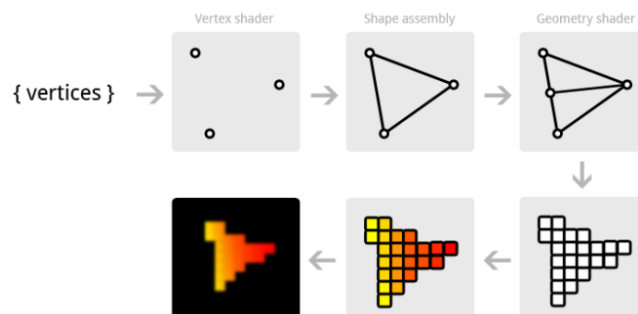
1 Задача

Переписать на шейдерах одну из лабораторных: 2, 3 или 6.

2 Основная теория

Шейдеры

- шейдер – программа, написанная на языке OpenGL Shader Language и выполняющаяся на GPU
 - вершинные шейдеры (vertex shader) – определение положения, цвета, текстурных координат;
 - геометрические шейдеры (geometry shader) – обработка примитивов;
 - фрагментные/пиксельные шейдеры (fragment/pixel shader) – управление растеризацией;
 - тесселяционные шейдеры (OpenGL 4+): (tessellation control shaders = hull shaders, tessellation evaluation shaders = domain shaders) – разбиение граней.



Создание окна и контекста OpenGL: библиотека GLFW

- контекст OpenGL: набор состояний, управляющих формированием изображения

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 2);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

glfwWindowHint(GLFW_RESIZABLE, GL_FALSE);
GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL", nullptr, nullptr);

attrib = glfwGetWindowAttrib(window, GLFW_CONTEXT_VERSION_MAJOR);
attrib = glfwGetWindowAttrib(window, GLFW_CONTEXT_VERSION_MINOR);
attrib = glfwGetWindowAttrib(window, GLFW_OPENGL_PROFILE);
```

Получение указателей на функции OpenGL: библиотека GLEW

- линковка проекта с библиотекой `glew32s.lib`
- подключение заголовочного файла до подключения заголовочного файла OpenGL и других библиотек, создающих контекст OpenGL

```
#define GLEW_STATIC
```

```
#include <GL/glew.h>
```

- вызов `glewInit()` после создания окна и контекста OpenGL

```
// Specify prototype of function typedef void
```

```
(*GENBUFFERS) (GLsizei, GLuint*);
```

```
// Load address of function and assign it to a function pointer
```

```
GENBUFFERS glGenBuffers =
```

```
(GENBUFFERS)wglGetProcAddress("glGenBuffers");
```

Постоянные параметры (Uniforms)

```
#version 150
```

```
uniform vec3 triangleColor;
```

```
out vec4 outColor;
```

```
void main()
```

```
{
```

```
    outColor = vec4(triangleColor, 1.0);
```

```
}
```

- `uniform` – параметр (переменная), значение которого одинаково для всех вызовов шейдера

Задание значения постоянного параметра

```
GLint uniColor = glGetUniformLocation(shaderProgram, "triangleColor");
```

```
glUniform3f(uniColor, 1.0f, 0.0f, 0.0f);
```

- uniform – параметр (переменная), значение которого одинаково для всех вызовов шейдера

Геометрические преобразования: основная программа

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
    // transformation (model)
glm::mat4 model;
GLint uniModel = glGetUniformLocation(shaderProgram, "model");
glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
    // camera (view)
glm::mat4 view = glm::lookAt( glm::vec3(1.2f, 1.2f, 1.2f),
    glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 0.0f, 1.0f) );
GLint uniView = glGetUniformLocation(shaderProgram, "view");
glUniformMatrix4fv(uniView, 1, GL_FALSE, glm::value_ptr(view));
    // projection
glm::mat4 proj = glm::perspective(45.0f, 800.0f / 600.0f, 1.0f, 10.0f);
GLint uniProj = glGetUniformLocation(shaderProgram, "proj");
glUniformMatrix4fv(uniProj, 1, GL_FALSE, glm::value_ptr(proj));
    // Draw a rectangle from the 2 triangles using 6 indices
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
```

Геометрические преобразования: вершинный шейдер

```
#version 150

in vec2 position;
in vec3 color;
in vec2 texcoord;

out vec3 Color;
out vec2 Texcoord;

uniform mat4 model;
uniform mat4 view;
uniform mat4 proj;

void main()
{
    Color = color;
    Texcoord = texcoord;
    gl_Position = proj * view * model * vec4(position, 0.0, 1.0);
}
```

3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```
1 import glfw
2 from OpenGL.GL import *
3 import numpy as np
4 from PIL import Image
5 import glm
6 import ctypes
7
8 angle_x, angle_y = 0.0, 0.0
9 scale = 0.7
10 wireframe = False
11 use_texture = True
12 light_position_toggle = False
13 light_diffuse_toggle = False
14 light_specular_toggle = False
15
16 a, b, c = 1.0, 0.6, 0.8
```

```

17 num_lat = 40
18 num_lon = 40
19
20 gravity = -9.8
21 y_pos = 1.0
22 y_velocity = 2.5
23 last_time = 0.0
24
25 def parametric_ellipsoid(theta, phi):
26     x = a * np.cos(theta) * np.cos(phi)
27     y = b * np.sin(theta)
28     z = c * np.cos(theta) * np.sin(phi)
29     return x, y, z
30
31 def generate_ellipsoid():
32     vertices = []
33     normals = []
34     texcoords = []
35     indices = []
36
37     for i in range(num_lat + 1):
38         theta = -np.pi / 2 + np.pi * i / num_lat
39         for j in range(num_lon + 1):
40             phi = 2 * np.pi * j / num_lon
41             x, y, z = parametric_ellipsoid(theta, phi)
42             vertices.extend([x, y, z])
43             nx = x / a
44             ny = y / b
45             nz = z / c
46             length = np.sqrt(nx**2 + ny**2 + nz**2)
47             normals.extend([nx/length, ny/length, nz/length])
48             texcoords.extend([j / num_lon, i / num_lat])
49
50     for i in range(num_lat):
51         for j in range(num_lon):
52             first = i * (num_lon + 1) + j
53             second = first + num_lon + 1
54             indices.extend([first, second, first + 1])
55             indices.extend([second, second + 1, first + 1])
56
57     return np.array(vertices, dtype=np.float32), np.array(normals, dtype
58 =np.float32), \
59         np.array(texcoords, dtype=np.float32), np.array(indices,
60 dtype=np.uint32)
61
62 def load_texture(path):

```

```

61     image = Image.open(path)
62     image = image.transpose(Image.FLIP_TOP_BOTTOM)
63     img_data = image.convert("RGB").tobytes()
64     width, height = image.size
65
66     texture_id = glGenTextures(1)
67     glBindTexture(GL_TEXTURE_2D, texture_id)
68
69     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height,
70                 0, GL_RGB, GL_UNSIGNED_BYTE, img_data)
71
72     glGenerateMipmap(GL_TEXTURE_2D)
73     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
74     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
75     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR)
76     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
77
78     return texture_id
79
80 def compile_shader(source, shader_type):
81     shader = glCreateShader(shader_type)
82     glShaderSource(shader, source)
83     glCompileShader(shader)
84     result = glGetShaderiv(shader, GL_COMPILE_STATUS)
85     if not(result):
86         raise RuntimeError(glGetShaderInfoLog(shader).decode())
87     return shader
88
89 def create_shader_program(vertex_source, fragment_source):
90     vertex_shader = compile_shader(vertex_source, GL_VERTEX_SHADER)
91     fragment_shader = compile_shader(fragment_source, GL_FRAGMENT_SHADER
)
92     program = glCreateProgram()
93     glAttachShader(program, vertex_shader)
94     glAttachShader(program, fragment_shader)
95     glLinkProgram(program)
96     result = glGetProgramiv(program, GL_LINK_STATUS)
97     if not(result):
98         raise RuntimeError(glGetProgramInfoLog(program).decode())
99     glDeleteShader(vertex_shader)
100    glDeleteShader(fragment_shader)
101    return program
102
103 vertex_shader_source = """
104 #version 330 core

```

```

105 layout (location = 0) in vec3 aPos;
106 layout (location = 1) in vec3 aNormal;
107 layout (location = 2) in vec2 aTexCoords;
108
109 out vec3 FragPos;
110 out vec3 Normal;
111 out vec2 TexCoords;
112
113 uniform mat4 model;
114 uniform mat4 view;
115 uniform mat4 projection;
116
117 void main()
118 {
119     FragPos = vec3(model * vec4(aPos, 1.0));
120     Normal = mat3(transpose(inverse(model))) * aNormal;
121     TexCoords = aTexCoords;
122     gl_Position = projection * view * vec4(FragPos, 1.0);
123 }
124 """
125
126 fragment_shader_source = """
127 #version 330 core
128 out vec4 FragColor;
129
130 in vec3 FragPos;
131 in vec3 Normal;
132 in vec2 TexCoords;
133
134 uniform vec3 lightPos;
135 uniform vec3 viewPos;
136 uniform sampler2D texture1;
137
138 void main()
139 {
140     float ambientStrength = 0.2;
141     vec3 ambient = ambientStrength * vec3(1.0);
142
143     vec3 norm = normalize(Normal);
144     vec3 lightDir = normalize(lightPos - FragPos);
145     float diff = max(dot(norm, lightDir), 0.0);
146     vec3 diffuse = diff * vec3(1.0);
147
148     float specularStrength = 0.5;
149     vec3 viewDir = normalize(viewPos - FragPos);
150     vec3 reflectDir = reflect(-lightDir, norm);

```



```

151     float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
152     vec3 specular = specularStrength * spec * vec3(1.0);
153
154     vec3 lighting = (ambient + diffuse + specular);
155     vec3 textureColor = texture(texture1, TexCoords).rgb;
156     FragColor = vec4(lighting * textureColor, 1.0);
157 }
158 """
159
160 def key_callback(window, key, scancode, action, mods):
161     global angle_x, angle_y, scale, wireframe, use_texture
162     global light_position_toggle, light_diffuse_toggle,
163     light_specular_toggle
164
165     if action == glfw.PRESS or action == glfw.REPEAT:
166         if key == glfw.KEY_UP:
167             angle_x -= 5
168         elif key == glfw.KEY_DOWN:
169             angle_x += 5
170         elif key == glfw.KEY_LEFT:
171             angle_y -= 5
172         elif key == glfw.KEY_RIGHT:
173             angle_y += 5
174         elif key == glfw.KEY_EQUAL:
175             scale += 0.1
176         elif key == glfw.KEY_MINUS:
177             scale -= 0.1
178         elif key == glfw.KEY_W:
179             wireframe = not wireframe
180             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else
181             GL_FILL)
182         elif key == glfw.KEY_T:
183             use_texture = not use_texture
184         elif key == glfw.KEY_1:
185             light_position_toggle = not light_position_toggle
186         elif key == glfw.KEY_2:
187             light_diffuse_toggle = not light_diffuse_toggle
188         elif key == glfw.KEY_3:
189             light_specular_toggle = not light_specular_toggle
190
191 def scroll_callback(window, xoffset, yoffset):
192     global scale
193     scale += 0.1 if yoffset > 0 else -0.1
194
195 def update_motion():
196     global y_pos, y_velocity, last_time

```

```

195
196     current_time = glfw.get_time()
197     delta_time = current_time - last_time
198     last_time = current_time
199
200     y_velocity += gravity * delta_time
201     y_pos += y_velocity * delta_time
202
203     if y_pos <= -1.0:
204         y_pos = -1.0
205         y_velocity = -y_velocity
206
207 def main():
208     global last_time
209
210     if not glfw.init():
211         raise Exception("GLFW can't be initialized")
212
213     window = glfw.create_window(800, 600, "Ellipsoid with Shaders", None
214     , None)
215     if not window:
216         glfw.terminate()
217         raise Exception("GLFW window can't be created")
218
219     glfw.make_context_current(window)
220     glfw.set_key_callback(window, key_callback)
221     glfw.set_scroll_callback(window, scroll_callback)
222
223     glEnable(GL_DEPTH_TEST)
224
225     vertices, normals, texcoords, indices = generate_ellipsoid()
226
227     VAO = glGenVertexArrays(1)
228     VBO = glGenBuffers(3)
229     EBO = glGenBuffers(1)
230
231     glBindVertexArray(VAO)
232
233     glBindBuffer(GL_ARRAY_BUFFER, VBO[0])
234     glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices,
235     GL_STATIC_DRAW)
236     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, ctypes.c_void_p
237     (0))
238     glEnableVertexAttribArray(0)
239
240     glBindBuffer(GL_ARRAY_BUFFER, VBO[1])

```

```

238     glBufferData(GL_ARRAY_BUFFER, normals.nbytes, normals,
GL_STATIC_DRAW)
239     glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, ctypes.c_void_p
(0))
240     glEnableVertexAttribArray(1)
241
242     glBindBuffer(GL_ARRAY_BUFFER, VBO[2])
243     glBufferData(GL_ARRAY_BUFFER, texcoords.nbytes, texcoords,
GL_STATIC_DRAW)
244     glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 0, ctypes.c_void_p
(0))
245     glEnableVertexAttribArray(2)
246
247     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO)
248     glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.nbytes, indices,
GL_STATIC_DRAW)
249
250     glBindVertexArray(0)
251
252     shader_program = create_shader_program(vertex_shader_source,
fragment_shader_source)
253     texture = load_texture(r"C:\Go\projects\Bmstu-projects\Grafic\lab6\
texture.bmp")
254
255     last_time = glfw.get_time()
256
257     while not glfw.window_should_close(window):
258         glfw.poll_events()
259         update_motion()
260
261         glClearColor(0.1, 0.1, 0.1, 1.0)
262         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
263
264         glUseProgram(shader_program)
265
266         projection = glm.perspective(glm.radians(45.0), 800/600, 0.1,
100.0)
267         view = glm.translate(glm.mat4(1.0), glm.vec3(0.0, 0.0, -4.0))
268         model = glm.mat4(1.0)
269         model = glm.rotate(model, glm.radians(angle_x), glm.vec3(1.0,
0.0, 0.0))
270         model = glm.rotate(model, glm.radians(angle_y), glm.vec3(0.0,
1.0, 0.0))
271         model = glm.translate(model, glm.vec3(0.0, y_pos, 0.0))
272         model = glm.scale(model, glm.vec3(scale, scale, scale))
273

```

```

274         light_pos = glm.vec3(2.0, 2.0, 2.0)
275         view_pos = glm.vec3(0.0, 0.0, 4.0)
276
277         glUniformMatrix4fv(glGetUniformLocation(shader_program, "model"),
278 , 1, GL_FALSE, glm.value_ptr(model))
279         glUniformMatrix4fv(glGetUniformLocation(shader_program, "view"),
280 , 1, GL_FALSE, glm.value_ptr(view))
281         glUniformMatrix4fv(glGetUniformLocation(shader_program, "
projection"), 1, GL_FALSE, glm.value_ptr(projection))
282         glUniform3fv(glGetUniformLocation(shader_program, "lightPos"),
283 , 1, glm.value_ptr(light_pos))
284         glUniform3fv(glGetUniformLocation(shader_program, "viewPos"), 1,
285 glm.value_ptr(view_pos))
286
287         if use_texture:
288             glActiveTexture(GL_TEXTURE0)
289             glBindTexture(GL_TEXTURE_2D, texture)
290             glUniform1i(glGetUniformLocation(shader_program, "texture1"), 0)
291
292             glBindVertexArray(VAO)
293             glDrawElements(GL_TRIANGLES, len(indices), GL_UNSIGNED_INT, None
294 )
295             glBindVertexArray(0)
296
297             glfw.swap_buffers(window)
298
299         glDeleteVertexArrays(1, [VAO])
300         glDeleteBuffers(3, VBO)
301         glDeleteBuffers(1, [EBO])
302         glDeleteProgram(shader_program)
303         glfw.terminate()
304
305     main()

```

Результат запуска представлен на рисунках 1– 2.

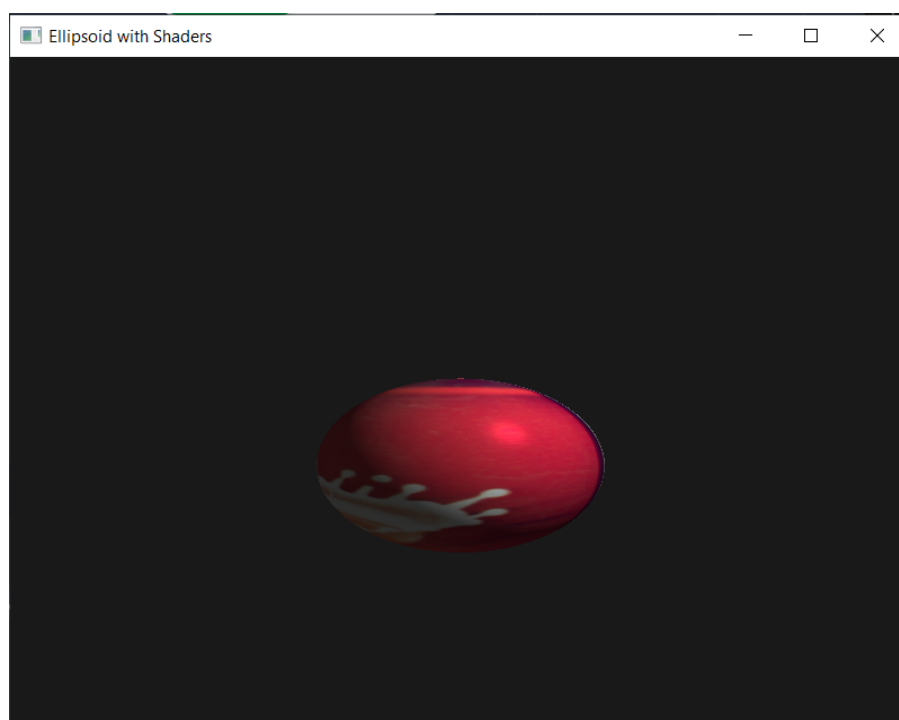


Рис. 1 — Эллипсоид с наложением текстуры

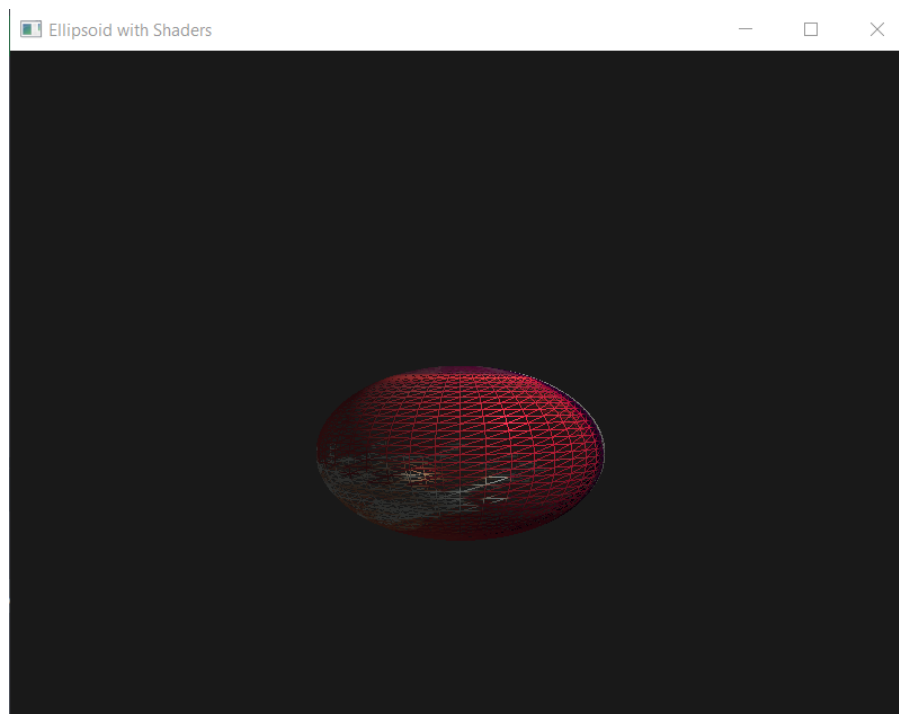


Рис. 2 — Каркасное отображение эллипсоида

4 Заключение

В ходе выполнения лабораторной работы №8 была реализована модернизация существующего OpenGL-приложения в лабораторной работе №6 путём переноса вычислений освещения и отображения материала с фиксированного функционала на программируемый с использованием шейдеров GLSL.

Были написаны и подключены вершинный и фрагментный шейдеры, реализующие модель освещения Фонга с поддержкой диффузной и зеркальной составляющих, а также текстурирование поверхности. Все необходимые трансформации (модельная, видовая и проекционная) были перенесены в вершинный шейдер с использованием библиотек `PyGLM` и `GLFW`.

Функциональность исходного проекта была полностью сохранена:

- Реализована отрисовка параметрического эллипсоида;
- Добавлены текстурирование, гравитация и отскок объекта;
- Сохранилось управление с клавиатуры: вращение, масштабирование, переключение каркасного режима.

Применение шейдеров позволило повысить гибкость, расширяемость и визуальную реалистичность сцены, а также приблизило проект к современным стандартам разработки в OpenGL.